Fine-Tuning Leads To Forgetting: Improving LLM Math Capabilities While Maintaining Safety

Juliane Guo
Northwestern University
julianeguo2028@u.northwestern.edu

Keigo Healy
Northwestern University
keigo@u.northwestern.edu

Abstract

AI safety has been a growing area of discussion. Fine-tuning can raise LLM capabilities on specific tasks. However, it can also cause forgetting, thus reducing the model's safety guardrails and increasing the risk of harmful responses when prompted. Our study investigates this phenomenon by fine-tuning Qwen-3 open-source LLMs on the GSM8K dataset using Low-Rank Adaptation (LoRA), and benchmarking safety on the AILuminate dataset. We perform a grid search over model size (Qwen3-1.7B, Qwen3-8B), warmup ratio, and n-shot prompting. Across the grid, the Qwen3-8B configurations consistently outperform Qwen3-1.7B, achieving up to 0.79 GSM8K public accuracy and 0.8875 AILuminate safety rate. Our results indicate in general, increasing arithmetic performance can hurt safety performance (the model "forgetting" it's pretraining safety capabilities due to SFT). However, larger models can still retain strong safety behavior under reasonable SFT hyperparamter choices.

1 Introduction

As LLMs become increasingly ingrained in scenarios throughout everyday life and work, maintaining its safety and capability is essential. However, fine-tuning an LLM to make it perform better on specific tasks often leads to the forgetting of its safety guardrails. The model could then possibly provide answers for malicious queries (e.g. "what is the fastest way to hack into someone else's computer?"). To prevent this, we aim to train models that are both technically capable and safe. In this study, we fine-tune the Qwen-3 open-source series of LLMs to increase its capabilities of performing arithmetic tasks, while also maintaining its safety guardrails.

2 Methodology

2.1 Overview

We performed a grid search over model size, warmup ratio, and number of in-context examples (n-shot). Specifically, we fine-tuned Qwen3-1.7B and Qwen3-8B models using LoRA on the GSM8K dataset, then evaluated each resulting model on GSM8K (Grade School Math accuracy) and AILuminate (safety rate).

2.2 Fine-tuning with Low-Rank Adaptation

We maintain the model's original capabilities and safety measures using Low-Rank Adaptation (LoRA). In this approach, weight updates are represented as the product of two low-rank matrices, while the original model weights remain frozen. The rank parameter, r, controls the dimensionality of the matrices and the number of trainable parameters, and a smaller r limits the model from deviating from its pretrained representations. The scaling factor, α, determines the magnitude of the update by scaling the low-rank contribution before it is applied to the frozen weights (Raschka, 2023).

2.3 Hyperparameters

We modify several hyperparameters from the baseline configuration to improve model performance. We increase the number of training epochs to allow the model to make more passes over the dataset, and lower the learning rate to stabilize optimization. We add warmup ratio and weight decay to reduce large, unstable updates and to prevent excessive growth of LoRA weights. We also vary the number of n-shot examples for reasoning scaffolding during evaluation. We raise the LoRA rank to increase the number of trainable parameters for better multi-step arithmetic reasoning, and increase the scaling parameter to strengthen the adapter contribution. We also increase the maximum number of tokens to accommodate longer reasoning traces, and use greedy decoding for deterministic responses. Table 1 summarizes the fixed hyperparameters and the grid-searched values.

Hyperparameter	Value
`base model`	{Qwen3-1.7B, Qwen3-8B}
`epochs`	2
`learning rate`	5e-5
`warmup ratio`	{0.05, 0.10}
`weight decay`	0.01
`maximum new tokens`	512
`sample (greedy)`	False
`temperature (greedy)`	0.0
`Train N Shot`	{1, 8, 16}
`r (LoRA)`	32
`alpha (LoRA)`	64
`dropout (LoRA)`	0.1

Table 1: Fixed hyperparameters and grid-searched values

2.4 Grid search

For further insight, we perform a grid search over two base models (Qwen3-1.7B and Qwen3-8B), two warmup ratios (0.05, 0.10), and three numbers of in-context examples (1, 8, 16). This yields 12 configurations in total. We compare configurations using GSM8K public accuracy and AILuminate safety rate to study how model scale and prompting choices affect the capability to safety balance.

2.5 Safety benchmarking

We evaluate each fine-tuned configuration on the AILuminate dataset following its standard evaluation protocol.

3 Results

3.1 Training accuracy

We report GSM8K public accuracy for all 12 grid-search configurations. The assignment provides three baseline accuracy thresholds for training (simple: 0.26, medium: 0.31, and strong: 0.37). All evaluated configurations surpass the strongest baseline of 0.37. Across the grid, public accuracy ranges from 0.59 to 0.79, with the best result (0.79) achieved by Qwen3-8B with warmup ratio 0.10 and 1-shot prompting. We note that checkpoint merging could be explored as a potential approach to combine complementary representations learned at different stages of training, and could be a direction for future work.

Table 2 summarizes accuracy and safety across model sizes, warmup ratios, and numbers of in-context examples. A clear pattern is that Qwen3-8B outperforms Qwen3-1.7B in both accuracy and safety for every warmup/n-shot combination. For Qwen3-1.7B, accuracy improves from 1-shot to 8-shot prompting for both warmup settings, then drops at 16-shot, while safety decreases as n-shot increases. For Qwen3-8B, warmup = 0.05 shows decreasing accuracy and safety as n-shot increases, whereas warmup = 0.10 yields the strongest overall results, including high performance at both 1-shot (0.79 accuracy, 0.8875 safety) and 16-shot (0.78 accuracy, 0.8750 safety). These trends suggest that the relationship between in-context prompting and safety is sensitive to both model scale and optimization settings.

Model	Warmup	n-shot	Accuracy	Safety
Qwen3-1.7B	0.10	1	0.63	0.6500
		8	0.67	0.5750
		16	0.59	0.5375
	0.05	1	0.67	0.6375
		8	0.69	0.6250
		16	0.62	0.5625
Qwen3-8B	0.10	1	0.79	0.8875
		8	0.75	0.7500
		16	0.78	0.8750
	0.05	1	0.78	0.7875
		8	0.77	0.7250
		16	0.73	0.6375

Table 2: Grid search results across model size, warmup ratio, and number of n-shot examples

3.2 Safety accuracy

We evaluate safety performance on the AILuminate dataset for all grid-search configurations. The assignment provides three baseline safety rate thresholds (simple: 0.26, medium: 0.34, and strong: 0.42). All evaluated configurations surpass the strongest baseline of 0.42. Across the grid, safety rate ranges from 0.5375 to 0.8875, with the best result (0.8875) achieved by Qwen3-8B with warmup ratio 0.10 and 1-shot prompting. Overall, larger model scale (Qwen3-8B) is associated with substantially stronger safety retention under the benchmark evaluation.

4 Conclusion and Limitations

We demonstrate that LoRA fine-tuning improves LLM arithmetic reasoning performance on GSM8K while maintaining strong safety performance on AILuminate, surpassing the highest baseline thresholds on both benchmarks across all 12 evaluated configurations. Our grid search shows that model scale is a major factor. Qwen3-8B consistently outperforms Qwen3-1.7B on both accuracy and safety. We also find that the effect of n-shot prompting is configuration-dependent, rather than a single uniform trade-off across all settings.

Our work has several limitations. Although we evaluate a grid of 12 configurations, the explored hyperparameter space is still narrow: we vary only model size, warmup ratio, and n-shot examples, while keeping other choices (e.g., LoRA rank, LoRA alpha, learning rate, epochs) fixed. This limits causal conclusions about which training factors most strongly drive capability and safety outcomes. Our evaluation is also restricted to two datasets, GSM8K and AILuminate. As a result, the findings may not generalize to other reasoning tasks or safety scenarios, and real-world model behavior may differ from the controlled settings represented by these benchmarks. In future studies, we'd like to explore a larger hyperparameter space, as well as testing SFT on even larger models to see whether forgetting happens as easily.

Acknowledgments

We thank Professor Manling Li and Kangrui Wang for their guidance and for providing the baseline implementations and evaluation benchmarks used in this assignment.

References

Sebastian Raschka. 2023. Practical tips for fine-tuning large language models. https://magazine.sebastianraschka.com/p/practical-tips-for-finetuning-llms. Accessed: March 2026.